



**UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET**



SEMINARSKI RAD

KVALITET I PREDOBRAĐA TEKSTUALNIH PODATAKA U MAŠINSKOM UČENJU

predobrada kratkih poruka za detekciju spama

Mentor: Prof. dr Aleksandar Stanimirović

Student: Ljiljana Stojičić, broj indeksa 2136

Niš, 2026.

Sadržaj

1. Uvod.....	4
2. Kvalitet tekstualnih podataka.....	4
2.1 Dužina dokumenata	5
2.2 Veličina rečnika i leksička raznovrsnost	5
2.3 Šum u tekstualnim podacima	5
2.4 Neuravnoteženost klasa kao aspekt kvaliteta.....	6
3. Tokenizacija.....	6
3.1 Tokenizacija na nivou reči.....	6
3.2 Tokenizacija na nivou rečenice	7
3.3 Subword (podreč) tokenizacija	7
4. Normalizacija teksta.....	8
4.1 Case folding	8
4.2 Unicode normalizacija	8
4.3 Zamena varijabilnih entiteta	8
4.4 Selektivno očuvanje diskriminativnog signala	9
5. Uklanjanje stop-reči	9
6. Stemming i lematizacija	10
7. Augmentacija podataka	11
7.1 EDA - Easy Data Augmentation tehnike.....	11
7.2 Back-translation	11
7.3 Augmentacija zasnovana na jezičkim modelima	11
8. Reprezentacije teksta.....	12
8.1 Bag-of-Words i TF-IDF.....	12
8.2 Statični word embeddings.....	12
8.3 Kontekstualni embeddings i transformer modeli	12
9. Uticaj predobrade na reprezentacije - teorijska sinteza.....	13
10. Metodologija praktičnog dela	14
11. Skup podataka.....	15
12. Priprema podataka i eksperimentalni dizajn	15
13. Rezultati.....	15
13.1 Kvalitet podataka - eksplorativna analiza	15
13.2 Predobrada teksta - rezultati.....	17
13.3 Uticaj predobrade na Bag-of-Words i TF-IDF.....	19

13.4 Uticaj predobrade na word embeddings	19
13.5 Uticaj predobrade na DistilBERT	20
13.6 Uporedna analiza i matrica konfuzije	20
13.7 Analiza značaja obeležja.....	22
14. Diskusija i preporuke.....	23
15. Zaključak	23
16. Literatura.....	25

1. Uvod

Kvalitet ulaznih podataka je granica koju nijedan algoritam mašinskog učenja ne može da pređe - model može da bude proizvoljno složen, ali njegova gornja granica performansi je određena time koliko su podaci na kojima uči tačni, potpuni i informativni. Kod strukturiranih (tabelarnih) podataka ova ideja je dobro razrađena kroz dimenzije poput tačnosti, kompletnosti i konzistentnosti pojedinačnih atributa. Kod nestrukturiranih podataka (teksta i slika) isti princip važi, ali se dimenzije kvaliteta moraju iznova definisati, jer ne postoji unapred zadata tabela atributa: „atributi” se moraju najpre izvući iz sirovog signala (niza karaktera ili piksela), i upravo taj korak izvlačenja je mesto gde se kvalitet i predobrada najviše prepliću [1].

Ovaj rad se bavi kvalitetom i predobradom nestrukturiranih podataka sa fokusom na tekst, konkretno, na kratke tekstualne poruke (SMS) i zadatak detekcije neželjene pošte (spama). Teorijski deo najpre definiše šta kvalitet znači kada je „atribut” zapravo niz reči promenljive dužine: dužinu dokumenata, veličinu rečnika, leksičku raznovrsnost i tipove šuma karakteristične za tekst. Zatim prelazi na tehnike predobrade - tokenizaciju, normalizaciju, uklanjanje stop-reči, stemming/lematizaciju i augmentaciju podataka - i na kraju povezuje sve to sa tri porodice reprezentacija teksta koje se koriste u praktičnom radu: klasične retke reprezentacije (Bag-of-Words, TF-IDF), statične vektorske reprezentacije (Word2Vec, FastText) i kontekstualne reprezentacije zasnovane na transformer arhitekturi (BERT/DistilBERT). Praktični deo demonstrira svaku od ovih tehnika na stvarnom skupu podataka i, metodologijom „sirov tekst naspram prethodno obrađenog teksta”, kvantitativno meri kakav uticaj predobrada ima na tačnost klasifikacije kroz sve tri porodice reprezentacija.

2. Kvalitet tekstualnih podataka

Za razliku od tabelarnih podataka, gde se kvalitet po pravilu procenjuje po koloni (opseg vrednosti, broj nedostajućih ćelija, tip podatka), kod teksta je „kolona” čitav dokument promenljive dužine sastavljen od diskretnih simbola. Kvalitet teksta se zato opisuje kroz statistike koje karakterišu ceo dokument ili ceo korpus, a ne pojedinačnu vrednost: dužinu dokumenata, veličinu rečnika, leksičku raznovrsnost i prisustvo različitih vrsta šuma. Sve četiri dimenzije su međusobno povezane - kratki dokumenti po pravilu imaju manji, ali gušće ponovljen rečnik, dok šum menja i dužinu (dodaje suvišne karaktere) i rečnik (uvodi varijante ispravnih reči) [1].

2.1 Dužina dokumenata

Dužina dokumenta izražena brojem karaktera, reči ili rečenica jeste najjednostavnija, ali daleko od trivijalne mere informativnosti teksta. Prekratak dokument često ne nosi dovoljno konteksta za pouzdanu odluku, dok predugačak uvodi šum. Kod modela sa fiksnom dužinom ulaza (transformeri) dužina direktno određuje da li će tekst biti skraćen (gubi se informacija) ili dopunjen (padding, povećava cenu bez dodavanja informacije). Sistematska razlika u dužini između klasa znači da dužina sama po sebi nosi diskriminativni signal koji jednostavni modeli mogu implicitno da nauče.

2.2 Veličina rečnika i leksička raznovrsnost

Veličina rečnika (vocabulary size) je broj jedinstvenih reči (tipova) u korpusu, dok je ukupan broj reči (tokena) zbir svih pojavljivanja. Odnos ova dva broja (Type-Token Ratio $TTR = |\text{tipovi}| / |\text{tokeni}|$) standardna je mera leksičke raznovrsnosti [1]. TTR ima važno metodološko ograničenje: osetljiva je na veličinu uzorka - kako korpus raste, verovatnoća ponavljanja već viđene reči raste brže od verovatnoće nove reči, pa TTR sistematski opada sa veličinom korpusa, nezavisno od stvarne leksičke raznovrsnosti. Zato je ispravno porediti TTR samo između uzoraka jednake dužine; u suprotnom se preporučuju mere manje osetljive na dužinu, poput MTLN (Measure of Textual Lexical Diversity).

Veličina rečnika direktno određuje dimenzionalnost klasičnih reprezentacija (BoW, TF-IDF), gde svaka jedinstvena reč postaje jedna dimenzija. Veliki rečnik znači veću, retko popunjenu matricu i veći rizik overfittinga na retke reči - centralna motivacija za rečnika bez gubitka bitne informacije.

2.3 Šum u tekstualnim podacima

Šum kod teksta obuhvata sve karaktere, reči ili strukture koje ne nose nameravano lingvističko značenje, već su posledica načina na koji je tekst nastao ili prikupljen. Vrsta šuma zavisi u velikoj meri od izvora teksta:

- Greške u kucanju i pravopisne greške - nastaju usled brzog kucanja, autokorekcije ili nepoznavanja pravopisa; posebno česti u neformalnoj mobilnoj komunikaciji.
- Spam i šablonski generisan sadržaj - tekst koji ima prepoznatljive strukturne obrasce (pozivi na akciju, brojevi telefona, valutni simboli, velika slova) koji odstupaju od prirodne raspodele jezika.

- HTML i markup ostaci - tekst preuzet sa veba često zadržava HTML tagove i entitete (&,) koji nisu deo čitljivog sadržaja.
- Emoji i emotikoni - simboli van standardnog alfabeta koji često nose sentiment informaciju, ali otežavaju tokenizaciju zasnovanu na ASCII skupu karaktera.
- OCR greške - sistematske zamene vizuelno sličnih karaktera (npr. „rn” → „m”, „0” → „O”), kvalitativno različit tip šuma od greške u kucanju jer nije posledica ljudske greške već ograničenja algoritma za prepoznavanje slike teksta.

Zajednička karakteristika svih ovih tipova šuma jeste da veštački uvećavaju veličinu rečnika i narušavaju pretpostavku da je ista reč uvek zapisana na isti način - pretpostavku na kojoj počivaju sve klasične reprezentacije teksta zasnovane na tačnom podudaranju stringova (BoW, TF-IDF). Prepoznavanje koji tip šuma dominira u konkretnom korpusu je zato preduslov za odabir odgovarajuće strategije normalizacije.

2.4 Neuravnoteženost klasa kao aspekt kvaliteta

Neuravnoteženost klasa je kod klasifikacije teksta izrazito česta - detekcija spama, prevare ili govora mržnje po prirodi imaju mnogo više „normalnih” nego „ciljanih” primera. Sa stanovišta kvaliteta podataka, ovo nije greška koju treba ispraviti čišćenjem, već svojstvo distribucije koje se mora uzeti u obzir pri podeli na trening/test skup (stratifikacija) i pri izboru mere uspešnosti - tačnost postaje obmanjujuća kada je jedna klasa retka, jer trivijalan model koji uvek predviđa većinsku klasu već postiže visoku tačnost. Zato se u praktičnom delu prati F1-mera manjinske klase.

3. Tokenizacija

Tokenizacija je postupak deljenja sirovog niza karaktera na manje, smislaone jedinice (tokene) koje dalje ulaze u sve naredne korake predobrade i modelovanja. Iako deluje kao trivijalan prvi korak, izbor granularnosti tokenizacije (reč, rečenica ili podreč) ima dubok uticaj na veličinu rečnika, sposobnost modela da rukuje rečima koje nije video u treningu (out-of-vocabulary, OOV problem) i na to koliko dobro reprezentacija odgovara morfologiji jezika [1].

3.1 Tokenizacija na nivou reči

Najintuitivnija strategija je deljenje teksta po razmacima i interpunkciji. Standardna konvencija za engleski jezik je Penn Treebank, koju implementira NLTK-ov ``word_tokenize`` [3]. Glavni

nedostatak: rečnik raste linearno sa brojem jedinstvenih reči, i svaka reč neviđena u treningu postaje nepoznat token (<UNK>) - model gubi svu informaciju o njoj, uključujući unutrašnju strukturu.

3.2 Tokenizacija na nivou rečenice

Deljenje na rečenice je preduslov za obradu na nivou veće jedinice od reči. Glavna poteškoća je razlikovanje tačke kao graničnika rečenice od tačke koja je deo skraćenice ili inicijala - problem koji zahteva statističke modele (npr. Punkt algoritam u NLTK-u), a ne prosto pravilo [3]. Kod kratkih poruka ovaj korak je manje kritičan jer poruka po pravilu i jeste jedna kratka rečenica.

3.3 Subword (podreč) tokenizacija

Rešenje problema OOV reči i eksplozije rečnika koje su usvojili gotovo svi savremeni transformer modeli jeste tokenizacija na nivou podreči (subword): umesto da svaka reč bude jedan token, redak ili nepoznat oblik se razlaže na manje, učestalije komade koje je model već video. Tri dominantna algoritma su:

- Byte-Pair Encoding (BPE) - iterativno spaja najčešći susedni par karaktera/podreči u novi simbol, počevši od pojedinačnih karaktera, sve dok rečnik ne dostigne zadatu veličinu; originalno predloženo za kompresiju podataka [4].
- WordPiece - konceptualno sličan BPE-u, ali umesto najčešćeg para bira spajanje koje najviše povećava verovatnoću trening korpusa pod jezičkim modelom; ovo je algoritam koji koriste BERT i njemu srodni modeli (uključujući DistilBERT) [2].
- Unigram / SentencePiece - polazi od velikog skupa kandidata za podreči i postepeno uklanja one čije uklanjanje najmanje povećava gubitak (loss) modela; SentencePiece dodatno tretira razmak kao običan karakter, što ga čini pogodnim i za jezike koji ne razdvajaju reči razmakom (kineski, japanski) [5].

Zajednički efekat sve tri metode je da čest rečnik ostaje kao celi tokeni, dok se redak ili nepoznat tekst (brojevi, žargon, imena, greške u kucanju, pa i reči iz drugih jezika) razlaže na manje, poznate komponente umesto da postane jedan nepoznat token. Cena ove fleksibilnosti je gubitak direktne čitljivosti (tokeni više nisu nužno cele reči) i potreba da model nauči da kombinuje informaciju raspoređenu na više podreči kako bi rekonstruisao značenje cele reči.

4. Normalizacija teksta

Normalizacija obuhvata skup transformacija koje svode različite površinske zapise iste informacije na jedinstven, kanonski oblik, čime se smanjuje veličina rečnika i broj tačaka u kojima model mora da uči da su dva različito zapisana tokena zapravo ista stvar.

4.1 Case folding

Case folding - svođenje teksta na mala slova - rešava problem da bi bez njega „Kuća”, „kuća” i „KUĆA” bile tri različite, nepovezane reči u rečniku. Cena ovog koraka je gubitak informacije koju veliko slovo nosi: razlikovanje vlastitih imenica od običnih reči (npr. „Apple” kompanija naspram „apple” voće) i signal emocionalnog naglašavanja (pisanje CELE REČI VELIKIM SLOVIMA, često korišćeno u spam-ovima da izrazi hitnost). Iz tog razloga se u praksi ovaj signal, kada je bitan za zadatak, često izdvaja kao posebno obeležje (npr. udeo reči napisanih velikim slovima) pre nego što se sam tekst svede na mala slova [1].

4.2 Unicode normalizacija

Isti vizuelno identičan karakter može u Unicode standardu biti predstavljen na više različitih binarnih načina. Na primer, slovo sa dijakritičkim znakom (é) može biti jedan složeni kodni znak ili kombinacija osnovnog slova i posebnog znaka za akcenat. Unicode standard definiše četiri normalizovane forme - NFC, NFD, NFKC i NFKD - koje ovakve ekvivalentne zapise svode na jedinstven binarni oblik, čime se osigurava da poređenje stringova (I tokenizacija i izgradnja rečnika) radi ispravno bez obzira na to kako je originalni tekst enkodiran [6]. „K” varijante (NFKC, NFKD) dodatno svode kompatibilne varijante karaktera (npr. razne tipove navodnika ili razmaka) na njihov osnovni oblik, što je posebno korisno kod teksta prikupljenog sa različitih platformi i uređaja, gde se isti karakter često enkodira na desetine različitih načina.

4.3 Zamena varijabilnih entiteta

Brojevi, URL adrese, adrese e-pošte i slični elementi imaju gotovo neograničen broj mogućih konkretnih vrednosti (svaki telefonski broj je drugačiji), pa bi njihovo direktno uključivanje u rečnik BoW/TF-IDF reprezentacije stvorilo ogroman broj tokena koji se pojavljuju samo jednom i ne nose generalizujuću informaciju. Uobičajena strategija je zamena čitave kategorije generičkim tokenom - svaki URL postaje token `URL`, svaki dugačak broj postaje token `NUM` - čime se čuva informacija da poruka SADRŽI link ili broj (što, kao što je pokazano u praktičnom delu ovog rada, može biti snažan diskriminativni signal), a da se pritom ne uvodi šum od hiljada jedinstvenih,

neponovljivih tokena. Ograničenje ovog pristupa je da mora unapred da definiše šta se smatra „brojem” ili „linkom” - kratke brojčane zamene korišćene u neformalnom govoru (npr. „2” umesto „to”, „4” umesto „for”) obično prolaze kroz ovaj filter neopaženo, jer se po dužini ne razlikuju od običnih, značenjskih brojeva.

4.4 Selektivno očuvanje diskriminativnog signala

Prethodna tri koraka normalizacije dele pretpostavku da je varijabilnost (velika slova, brojevi, ponovljena interpunkcija) šum koji treba ukloniti. Ta pretpostavka nije univerzalno tačna: kod zadataka poput detekcije spama, upravo takva „varijabilnost” (CELA VELIKA SLOVA, simboli valuta, nizovi uzvičnika) nije slučajan šum, već sistematski obrazac koji karakteriše jednu od klasa. Kada normalizacija ne razlikuje ova dva slučaja, rizikuje se da se zajedno sa šumom obriše i koristan signal.

Praktično rešenje je da se normalizacija podeli u dva toka: informacija koja je pravi šum (tačan telefonski broj, tačan iznos) svodi se na generičku oznaku ili briše, dok se činjenica da je obrazac bio prisutan beleži kao posebno, eksplicitno obeležje, umesto da nestane zajedno sa detaljima. Ovaj princip “normalizuj detalj, sačuvaj obrazac” direktno je testiran u praktičnom delu kroz poređenje dve varijante predobrade: jedne koja šum briše i druge koja ga svesno prevodi u obeležja.

5. Uklanjanje stop-reči

Stop-reči su visokofrekventne funkcionalne reči (predlozi, veznici, zamenice) koje se pojavljuju u gotovo svakom dokumentu nezavisno od teme, pa nose relativno malo diskriminativne informacije u poređenju sa svojom učestalošću [7]. Lista je po pravilu kratka i fiksna (NLTK-ova lista za engleski ima ~180 unosa [3]), pa njeno uklanjanje bitno ne menja veličinu rečnika, ali značajno smanjuje broj tokena po dokumentu i težinu visokofrekventnih reči u BoW/TF-IDF.

Uklanjanje stop-reči nosi i realan rizik: negacije („not”, „no”, „never”) su na fiksnoj listi, a njihovo uklanjanje može doslovno preokrenuti sentiment rečenice. Kod modela koji uče sopstvenu reprezentaciju konteksta (embeddings, transformeri) uklanjanje stop-reči je često nepotrebno ili štetno, jer ti modeli koriste funkcionalne reči kao deo konteksta iz kog uče značenje.

6. Stemming i lematizacija

Cilj oba postupka je svođenje različitih morfoloških oblika iste reči („run”, „runs”, „running”, „ran”) na jedan zajednički oblik, kako bi model tretirao sve njih kao istu informacionu jedinicu umesto kao četiri nepovezana tokena, ali se razlikuju po tome kako do tog zajedničkog oblika dolaze.

Stemming mehanički odseca sufikse prema unapred definisanom skupu pravila, bez rečnika i bez razumevanja gramatičke uloge reči u rečenici. Najrasprostranjeniji algoritam je Porterov stemmer, objavljen 1980. godine, koji kroz pet sekvencijalnih faza pravila uklanja najčešće engleske sufikse (npr. „-ing”, „-ed”, „-ational” → „-ate”) [8]. Prednost stemminga je brzina i jednostavnost, ne zahteva rečnik niti gramatičku analizu, a mana je što proizvedeni „koren” često nije stvarna reč engleskog jezika (npr. „studies” → „studi”), što otežava čitljivost rezultata, iako to ne mora da bude problem ako se koren koristi isključivo kao interni identifikator obeležja u modelu, a ne kao tekst koji čita čovek.

Lematizacija umesto mehaničkog odsecanja koristi rečnik i morfološku analizu da pronađe pravi osnovni, rečnički oblik reči (lemu), npr. „better” → „good”, „was” → „be”. Najčešće korišćen resurs za engleski jezik je WordNet, leksička baza koja grupiše reči u skupove sinonima i beleži njihove osnovne oblike [9]. Ključno ograničenje lematizacije je da tačnost direktno zavisi od toga da li je algoritmu unapred poznata gramatička vrsta reči (part-of-speech, POS) - isti string može imati različitu lemu zavisno od toga da li je glagol ili imenica (npr. „saw” kao prošlo vreme „see” naspram imenice „testera”). Bez POS oznake, lematizatori pretpostavljaju podrazumevanu vrstu (najčešće imenicu), pa glagolski nastavci poput „-ing”/„-ed” ostaju nepromenjeni, koje savremeni alati poput spaCy rešavaju integrisanjem POS tagera u pipeline lematizacije [10]. Sama tačnost POS tagovanja pritom zavisi od rečeničnog konteksta - tager primenjen na izolovanu reč greši češće nego u rečenici.

Izbor između stemminga i lematizacije je kompromis između računске jednostavnosti i lingvističke preciznosti: stemming je brži i agresivniji u redukciji veličine rečnika, dok je lematizacija sporija (zahteva rečnik i, za pun efekat, POS oznake) ali čuva čitljivost i preciznije pogađa stvarno značenje osnovnog oblika. Kod klasičnih retnih reprezentacija (BoW/TF-IDF), gde je jedini cilj smanjenje dimenzionalnosti bez potrebe da rezultat bude čitljiv, oba pristupa su podjednako legitimna; kod savremenih modela zasnovanih na neuronskim mrežama sa subword

tokenizacijom, oba koraka su često suvišna, jer subword tokenizacija već implicitno hvata deljene korene reči kroz zajedničke podreči.

7. Augmentacija podataka

Augmentacija podataka označava veštačko generisanje dodatnih trening primera iz postojećih. Kod teksta je ovo tehnički zahtevnije nego kod slika: geometrijska transformacija slike skoro uvek proizvodi validnu sliku istog objekta, dok proizvoljna izmena reči u rečenici lako može da uništi značenje ili proizvede gramatički neispravnu rečenicu.

7.1 EDA - Easy Data Augmentation tehnike

Wei i Zou (2019) predlažu četiri jednostavne, računski jeftine operacije: sinonimsku zamenu, nasumično umetanje, nasumičnu zamenu mesta i nasumično brisanje reči, koje, primenjene u malom broju po rečenici, u proseku poboljšavaju tačnost klasifikacije, sa najvećim dobitkom na malim skupovima podataka [11]. Glavno ograničenje je da zamena sinonimima iz WordNet-a ne uzima u obzir kontekst - reč može biti zamenjena sinonimom pogrešnog značenja, što unosi semantički nekoherentne rečenice u trening skup.

7.2 Back-translation

Back-translation (prevod-i-povratak) generiše parafraze prevođenjem rečenice na drugi jezik i nazad - dvostruki prevod obično daje rečenicu sličnog značenja, ali drugačije formulacije, prirodniju parafrazu nego proste operacije na nivou reči. Tehnika je prvobitno razvijena za poboljšanje mašinskog prevođenja zasnovanog na neuronskim mrežama (NMT) korišćenjem jednojezičkih podataka [12], a pokazala se korisnom i za augmentaciju klasifikacije teksta. Cena je računska (potreban je model za prevođenje u oba smera) i rizik sistematske promene stila pri prekomernoj upotrebi.

7.3 Augmentacija zasnovana na jezičkim modelima

Savremeniji pristup koristi maskirane ili generativne jezičke modele (porodica BERT) da predlože zamenu reči na osnovu konteksta cele rečenice, umesto fiksnog rečnika sinonima. Rizik od pogrešnog značenja je znatno manji nego kod EDA, ali cena je veća računska složenost, forward-pass kroz neuronsku mrežu za svaku augmentaciju.

8. Reprezentacije teksta

Da bi tekst uopšte mogao da bude ulaz modelu mašinskog učenja, mora se preslikati u numerički oblik. Istorijski, ovo preslikavanje je prošlo kroz tri generacije pristupa, od kojih svaka na drugačiji način balansira jednostavnost, dimenzionalnost i sposobnost da uhvati značenje reči u kontekstu.

8.1 Bag-of-Words i TF-IDF

Bag-of-Words (BoW) predstavlja dokument kao vektor učestalosti reči iz rečnika korpusa, potpuno zanemarujući redosled reči - otuda i naziv „vreća reči”. Svaka dimenzija vektora odgovara jednoj reči, a njena vrednost broju pojavljivanja [7]. TF-IDF (Term Frequency – Inverse Document Frequency) unapređuje ovu ideju dodeljivanjem težina: težina reči raste sa učestalošću u datom dokumentu (TF), ali opada ukoliko se reč pojavljuje u mnogo dokumenata korpusa (IDF), čime se automatski umanjuje uticaj reči koje su česte u celom korpusu [7][13]. Oba pristupa dele iste osobine: matrica je retka i visoko dimenzionalna, neosetljiva na redosled reči, i ne poznaje semantičku sličnost - sinonimi „automobil” i „kola” su dve potpuno nepovezane dimenzije.

8.2 Statični word embeddings

Word embeddings rešavaju problem semantičke izolovanosti reči tako što svaku reč preslikavaju u vektor male dimenzionalnosti (tipično 100–300), obučen tako da reči sa sličnim značenjem dobiju blizu pozicionirane vektore. Word2Vec uči vektore kroz CBOW (predviđanje centralne reči iz okolnih) ili Skip-gram (obrnuto) [14]. GloVe direktno faktorizuje globalnu matricu koučestalosti reči [15]. Oba dele istu manu: reč koja se ne pojavljuje u trening korpusu (OOV) nema vektor. FastText to rešava predstavljanjem reči kao skupa n-grama na nivou karaktera - vektor cele reči je zbir vektora njenih n-grama, pa model može da proceni razuman vektor i za neviđenu reč [16]. Zajednička slabost svih statičnih embeddinga: svaka reč dobija tačno jedan vektor nezavisno od konteksta.

8.3 Kontekstualni embeddings i transformer modeli

Transformer arhitektura zamenjuje rekurentnu obradu sekvence mehanizmom pažnje (self-attention), koji svakoj reči dozvoljava da direktno dodeljuje težinu informacijama iz svih ostalih reči u rečenici, nezavisno od rastojanja. BERT kroz prethodno obučavanje na ogromnim količinama teksta (maskirani jezički model) uči duboke, kontekstualne reprezentacije [2]. DistilBERT je manja, destilovana verzija BERT-a koja postiže ~97% njegove sposobnosti razumevanja jezika uz ~40% manje parametara i 60% veću brzinu [17]. Transformer modeli se

koriste na dva načina: fino podešavanje (fine-tuning celog modela) ili ekstrakcija obeležja (model „zamrznut”, koristi se samo za generisanje vektora za jednostavniji klasifikator) - pristup korišćen u praktičnom delu ovog rada.

9. Uticaj predobrade na reprezentacije - teorijska sinteza

Prethodna poglavlja su predobradu i reprezentacije teksta predstavila odvojeno; u praksi njihova interakcija najviše određuje krajnji rezultat. Klasične retke reprezentacije (BoW, TF-IDF) direktno „vide” svaki tačan string u rečniku - normalizacija, uklanjanje stop-reči i lematizacija im pomažu smanjenjem broja redundantnih dimenzija, pa se očekuje da predobrada ovim reprezentacijama u proseku koristi. Postoji, međutim, i rizik suprotnog smera: ako normalizacija ne razlikuje šum od diskriminativnog obrasca, ista operacija može ukloniti signal koji je model mogao da iskoristi. Zato se u praktičnom delu porede ne dve, već tri: sirov tekst, agresivna predobrada (briše šum) i pažljiva predobrada (čuva obrazac kao obeležje) - čime se hipoteza direktno testira, umesto da se pretpostavlja.

Statični word embeddings (Word2Vec, FastText) su osetljivi na veličinu korpusa na kom se treniraju. Ako se treniraju „od nule” na malom korpusu, ne postoji dovoljno ponavljanja konteksta da bi model naučio pouzdane vektore, nezavisno od predobrade; tada je predobrada sekundarni faktor u odnosu na fundamentalni nedostatak podataka.

Kontekstualni transformer modeli (BERT, DistilBERT) su poseban slučaj: pošto su prethodno obučeni na sirovom, prirodnom tekstu, agresivna klasična predobrada uklanja upravo onu vrstu konteksta na koju se samopažnja oslanja. Teorijska preporuka je zato minimalna predobrada - osloniti se na subword tokenizaciju umesto na klasičan pipeline dizajniran za retke reprezentacije.

Reprezentacija	Očekivani efekat predobrade	Glavni rizik
Bag-of-Words / TF-IDF	Uglavnom koristi - smanjuje redundantne dimenzije	Gubitak diskriminativnih signala (brojevi, velika slova) ako se agresivno ukalanjaju
Word2Vec / FastText (trenirano od nule)	Sekundaran u odnosu na veličinu korpusa	Nedovoljno podataka daje nepouzdan vektore bez obzira na predobradu
BERT / DistilBERT (kontekstualni)	Često šteti	Uklanjanje stop-reči i lematizacija uništavaju kontekst na koji se samopažnja oslanja

Tabela 1. Uticaj predobrade na reprezentacije

10. Metodologija praktičnog dela

Praktični deo rada ima za cilj da empirijski proveriti teorijska očekivanja na konkretnom, dobro poznatom zadatku - binarnoj klasifikaciji kratkih tekstualnih poruka na neželjenu poštu (spam) i legitimne poruke (ham). Metodologija je organizovana kroz sledeće korake, koji direktno prate strukturu teorijskog dela:

- Eksplorativna analiza i provera kvaliteta (nedostajuće vrednosti, duplikati, raspodela klasa, dužina dokumenata, veličina rečnika, leksička raznovrsnost, najfrekventnije reči po klasi, tipovi šuma).
- Demonstracija tokenizacije na tri nivoa granularnosti (reč, rečenica, podreč) i normalizacije (Unicode, case folding, zamena URL-ova/brojeva) na ilustrativnim primerima sa emoji/HTML šumom.
- Uklanjanje stop-reči, i poređenje stemminga, naivne i POS-svesne lematizacije po veličini rečnika koji svaki proizvodi.
- Demonstracija sve četiri EDA operacije i back-translation augmentacije nad manjinskom (spam) klasom.
- Izgradnja tri verzije korpusa - sirove, agresivno očišćene (šum se briše) i pažljivo pripremljene (šum postaje eksplicitno obeležje) i identična stratifikovana podela na trening/test skup.
- Treniranje klasifikatora nad sve tri verzije korpusa, kroz tri porodice reprezentacija: BoW/TF-IDF (Naivni Bajes, logistička regresija), Word2Vec/FastText (trenirani na korpusu) i DistilBERT (zamrznut ekstraktor obeležja).
- Poređenje rezultata po F1-meri, analiza matrice konfuzije najboljeg modela, i analiza značaja obeležja.

Sav praktični rad izveden je u Python-u, uz pandas/NumPy, NLTK (tokenizacija, stop-reči, POS, stemming, lematizacija), gensim (Word2Vec/FastText), Hugging Face transformers i PyTorch (DistilBERT, MarianMT) i scikit-learn (vektorizacija, klasifikatori, mere uspešnosti). Kompletan kod je dat u Jupyter notebook-u koji prati isti redosled koraka kao ovo poglavlje.

11. Skup podataka

Za praktični deo korišćen je SMS Spam Collection - javno dostupan skup od 5.572 SMS poruka na engleskom jeziku, ručno obeleženih kao „ham” ili „spam”, koji su sakupili i objavili Almeida i Hidalgo. Skup je izabran jer poruke su kratke, pisane neformalnim jezikom, i sadrže prirodne primere gotovo svih tipova šuma. Klase su prirodno neuravnotežene što skup čini pogodnim i za ilustraciju uticaja neuravnoteženosti na izbor mere uspešnosti.

12. Priprema podataka i eksperimentalni dizajn

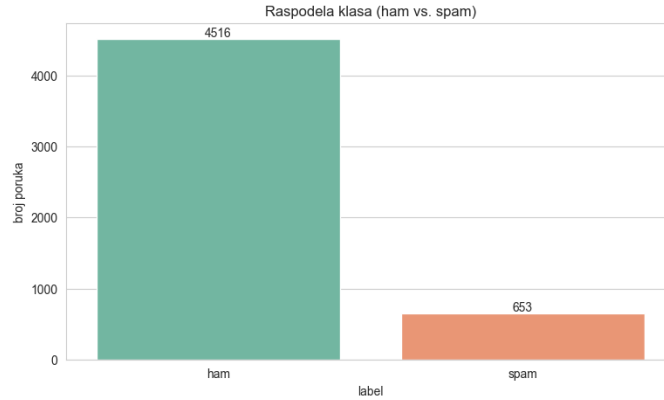
Pre same lingvističke obrade sprovedena je provera osnovnog kvaliteta skupa: nema nedostajućih vrednosti, ali je pronađeno 403 potpuno identične poruke (7,2% skupa), verovatno posledica ponovljenih reklamnih šablona kod pošiljalaca spama. Duplikati su uklonjeni pre podele na trening i test skup - u suprotnom bi ista poruka mogla da se nađe u oba skupa (curenje podataka). Nakon uklanjanja duplikata skup broji 5.169 poruka.

Da bi se izmerio čist efekat predobrade, trening i test skup formirani su samo jednom, stratifikovanom podelom 80:20 (4.135/1.034 poruka), korišćenom identično za sve reprezentacije i sve tri verzije teksta. Sirova verzija primenjuje samo case folding. Agresivna predobrada prolazi kroz pun klasičan pipeline (Unicode normalizacija, proširivanje skraćenica, zamena URL-ova/brojeva, uklanjanje interpunkcije, stop-reči, lematizacija), pri čemu se velika slova, valuta i ponovljena interpunkcija tretiraju kao šum i brišu. Pažljiva predobrada prolazi kroz identičnu lingvističku obradu, ali pre svođenja na mala slova beleži prisustvo velikih slova/valute/ponovljene interpunkcije kao eksplicitne oznake (`ALLCAPSFLAG`, `CURRENCYFLAG`, `MULTIEXCLAIM`).

13. Rezultati

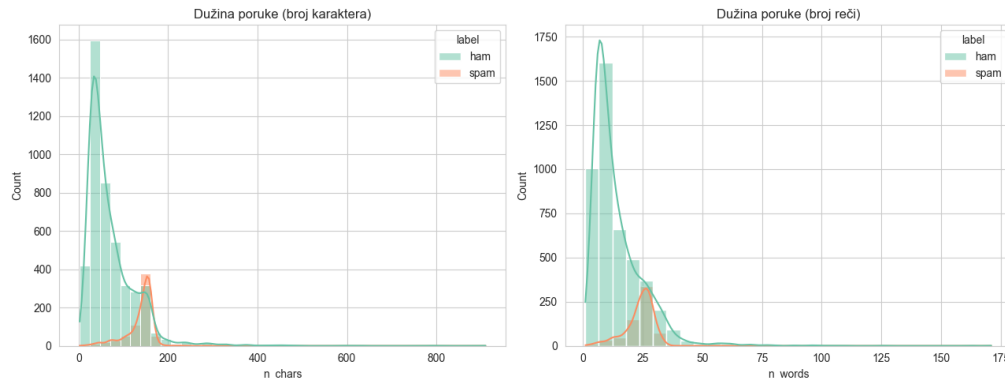
13.1 Kvalitet podataka - eksplorativna analiza

Nakon uklanjanja duplikata, skup sadrži 4.516 ham poruka (87,4%) i 653 spam poruke (12,6%), odnos otprilike 7:1, što potvrđuje očekivanu neuravnoteženost klasa i opravdava praćenje F1-mere umesto proste tačnosti u svim narednim eksperimentima.



Slika 1. Raspodela klasa (ham naspram spam) nakon uklanjanja duplikata.

Analiza dužine poruka pokazuje jasnu razliku između klasa: spam poruke su dosledno duže i uniformnije - prosečno 137,7 karaktera (std 29,8, opseg 13–223) i 23,7 reči, blizu granice od 160 karaktera po SMS-u. Ham poruke su raznovrsnije - prosek 70,9 karaktera, ali sa ogromnim rasponom (std 56,7; od 2 do 910) i 14,2 reči u proseku. Raspodela ham poruka je izrazito desno zakošena sa vrhom kod kratkih poruka, dok spam formira uzak brežuljak koncentrisan između 120 i 160 karaktera. Dužina poruke je, dakle, sama po sebi već korisna diskriminativna karakteristika.

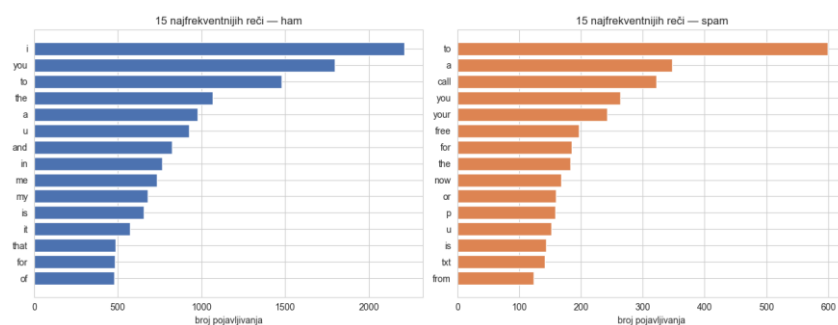


Slika 2. Raspodela dužine poruka po klasi (broj karaktera i broj reči).

Ham poruke ukupno sadrže 63.906 tokena sa rečnikom od 6.913 jedinstvenih reči (TTR = 0,1082), dok spam poruke sadrže 14.832 tokena sa rečnikom od 2.127 reči (TTR = 0,1434). Deluje neintuitivno da spam ima veću leksičku raznovrsnost, ali ovo je upravo metodološka zamka TTR mere: manji spam korpus automatski dobija veći TTR jer ima manje prilika da ponovi već viđenu reč. Ispravno poređenje bi zahtevalo uzorke jednake dužine ili robusniju meru (npr. MTLD).

Slika 3 prikazuje 15 najfrekventnijih reči po klasi, sa tačnim brojem pojavljivanja. Kod ham poruka sve su čisto funkcionalne („i”, „you”, „to”). Kod spam poruka, odmah posle funkcionalnih

reči probijaju se karakteristične spam reči - „call”, „free”, „now”, „txt”- i upravo se ove reči ispostavljaju kao neki od najjačih pojedinačnih prediktora spama.



Slika 3. Petnaest najfrekventnijih reči po klasi, sa tačnim brojem pojavljivanja.

Analiza šuma na ovom skupu pokazuje sledeću sliku: 14,4% poruka sadrži reč napisanu celu velikim slovima (naglašavanje, tipično za spam - „WINNER”, „FREE”), 10,6% sadrži dug niz cifara (telefonski brojevi, premium brojevi), 4,6% sadrži simbol valute, 2,5% sadrži URL, a 1,8% ima ponovljenu interpunkciju. Ovi obrasci nisu ravnomerno raspoređeni između klasa, gotovo su isključivo obeležje spam poruka. Skup je nastao pre masovne upotrebe emotikona pa ne sadrži emoji ni HTML entitete. Ilustrativni primer u notebook-u pokazuje da naivna normalizacija emoji simbole u potpunosti uklanja, dok HTML oznake (`<b>`, `&nbsp;`) uklanja samo delimično - interpunkcija nestaje, ali imena tagova/entiteta („b”, „nbsp”) ostaju kao usamljeni tokeni, jer generička normalizacija ne prepoznaje HTML sintaksu.

Tip šuma	Udeo poruka
Reč napisana velikim slovima (3+ slova)	14,4%
Dug niz cifara (4+ cifre)	10,6%
Simbol valute (£/\$/€)	4,6%
URL / web adresa	2,5%
Ponovljena interpunkcija (!!, ??)	1,8%

Tabela 2. Tipovi šuma i njihova zastupljenost u skupu podataka

13.2 Predobrada teksta - rezultati

Tokenizacija istog primera spam poruke na sva tri nivoa granularnosti potvrđuje teorijska očekivanja: word-level tokenizacija tretira brojeve i skraćenice kao zasebne, glomazne celine, dok subword (WordPiece) tokenizacija istu poruku deli na znatno više, kraćih podreči, nepoznata reč se razlaže na poznate komponente (npr. „wkly” postaje „w” + „##k” + „##ly”) što direktno ilustruje zašto subword pristup rešava problem OOV reči.

Uklanjanje stop-reči smanjuje rečnik sa 7.652 na 7.514 jedinstvenih reči, svega ~1,8%. NLTK-ova lista ima svega ~180 unosa, pa uklanjanje stop-reči praktično ne menja veličinu rečnika; njegov stvarni efekat je na broj tokena po dokumentu i na težine u BoW/TF-IDF.

Poređenje stemminga i lematizacije pokazuje tačno najavljeno ograničenje: uz eksplicitnu oznaku glagola, lematizator tačno pogađa osnovni oblik („studies” → „study”), ali uz podrazumevanu (imeničku) pretpostavku, glagolski nastavci ostaju netaknuti. Rečnik se sa 7.514 smanjuje na 6.195 posle stemminga (–17,6%), ali samo na 7.000 posle naivne lematizacije (–6,8%).

Kada se svakoj reči prvo odredi stvarna gramatička vrsta POS tagerom, ispravljaju se slučajevi koje je naivna verzija promašila: „wolves” → „wolf”, „universities” → „university”, dok primer „better” → „well” (umesto „good”) ilustruje granicu metode - tager primenjen na izolovanu reč pogrešno je prepoznao prilog umesto prideva. Na celom korpusu (gde tager ima pravi rečenični kontekst), rečnik se dodatno smanjuje sa 7.000 na 6.580 (–6%), POS-svesna lematizacija je istovremeno i tačnija i efikasnija.

Verzija rečnika	Veličina rečnika	Promena
Posle normalizacije (pre uklanjanja stop-reči)	7.652	—
Posle uklanjanja stop-reči	7.514	-1,8%
Posle stemminga	6.195	-17,6%
Posle naivne lematizacije (bez POS oznaka)	7.000	-6,8% (u odnosu na tekst posle uklanjanja stop-reči)
Posle POS-svesne lematizacije	6.580	-6,0% (u odnosu na naivnu lematizaciju)

Tabela 3. Veličina rečnika kroz korake predobrade

Demonstracija sve četiri EDA operacije nad istom spam porukom potvrđuje teorijski najavljeno ograničenje sinonimske zamene: dok su neke zamene semantički razumne (npr. „photo” → „photograph”), druge biraju pogrešno značenje iz WordNet-a bez uzimanja konteksta u obzir - skraćenica „c” je zamenjena sa „ascorbic acid” (vitamin C), a „p” sa „phosphorus”. Preostale tri operacije (brisanje, zamena mesta, umetanje) su jeftinije za izračunavanje, ali menjaju rečenicu na podjednako grub način - brisanje ume da ukloni i trećinu reči iz kratke poruke.

Back-translation, primenjen na četiri poruke preko francuskog jezika i nazad (MarianMT), pokazuje oba lica ove tehnike. Na relativno standardnom engleskom prevod je gotovo besprekoran: „You've won tkts to the EURO2004 CUP FINAL or £800 CASH...” vraća se kao „You have won tkts at EURO2004 FINAL CUP or £800 CASH...” - ista poruka, prirodno

preformulisana, bez upotrebe WordNet-a i bez rizika pogrešnog značenja koji smo videli kod EDA tehnika. Na izrazito neformalnom, skraćenicama prepunom tekstu prevod je znatno nepouzdaniji. Zaključak: back-translation je pouzdaniji za standardni jezik nego za žargon kratkih poruka, upravo onaj registar koji dominira u ovom skupu podataka.

13.3 Uticaj predobrade na Bag-of-Words i TF-IDF

Tabela ispod poredi F1-meru za spam klasu, za dva klasifikatora (Naivni Bajes i logistička regresija) nad dve reprezentacije (BoW i TF-IDF), na sve tri verzije teksta: sirovog, agresivno predobrađenoj (šum se briše) i pažljivo predobrađenoj (šum se svodi na eksplicitna obeležja).

Reprezentacija	Sirov tekst (F1)	Agresivna predobrada (F1)	Pažljiva predobrada (F1)
BoW + Naivni Bajes	0,9308	0,9208	0,9398
BoW + Logistička regresija	0,9024	0,9098	0,9231
TF-IDF + Naivni Bajes	0,8559	0,8945	0,8898
TF-IDF + Logistička regresija	0,8462	0,7928	0,8142

Tabela 4. Uticaj predobrade na Bag-of-Words i TF-IDF reprezentacije

Ovde se prvi put jasno vidi da predobrada može da pomogne: kod obe BoW kombinacije, pažljiva predobrada je najbolja od sve tri verzije - bolja i od sirovog teksta i od agresivne predobrade (Naivni Bajes: 0,9398 naspram 0,9208; LogReg: 0,9231 naspram 0,9098). Kod TF-IDF-a slika je složenija: pažljiva predobrada nadmašuje sirov tekst kod Naivnog Bajesa, dok kod logističke regresije sirov tekst ostaje najbolji. Odgovor na „koja predobrada je najbolja” zavisi i od konkretnog klasifikatora, ne samo od toga koliko se signala sačuva.

13.4 Uticaj predobrade na word embeddings

Svih šest kombinacija Word2Vec i FastText, na sve tri verzije teksta daju $F1(\text{spam}) = 0$: model se u potpunosti „predao” i predviđa isključivo klasu ham (tačnost ~87,3%, tačno udeo ham poruka). Ni pažljiva predobrada ovde ništa ne menja, 4.135 trening poruka jednostavno nije dovoljno da se nauči pouzdan vektor ni za koji token, uključujući novododate oznake. Ovo potvrđuje da problem nije bio koji tekst se daje, već koliko ga ima. Veličina korpusa je presudniji faktor od izbora predobrade. Samostalno treniranje embeddinga ima smisla samo uz dovoljno velik korpus [14][16]; na malim korpusima bolji izbor su unapred trenirani vektori ili kontekstualni modeli.

Isti postupak ponovljen je sa unapred treniranim vektorima (Word2Vec na Google News korpusu i FastText na Wikipediji i vestima, oba 300-dimenzionalna). Rezultat je nedvosmislen, F1(spam) skače sa 0 na 0,785–0,794 (Word2Vec) i 0,644–0,707 (FastText) u sve tri verzije teksta, čime se potvrđuje da je problem bio u veličini korpusa na kom su vektori trenirani. Zanimljivo, kod unapred treniranih vektora predobrada pomaže dosledno i monotono (pažljiva > agresivna > sirov, kod oba modela) suprotno DistilBERT-u, jer statični vektori, za razliku od kontekstualnog modela sa mehanizmom pažnje, nemaju sopstveni mehanizam da zanemare nepotrebne funkcionalne reči, pa im njihovo uklanjanje realno koristi. Oba pretrenirana modela i dalje zaostaju za najboljim rezultatima.

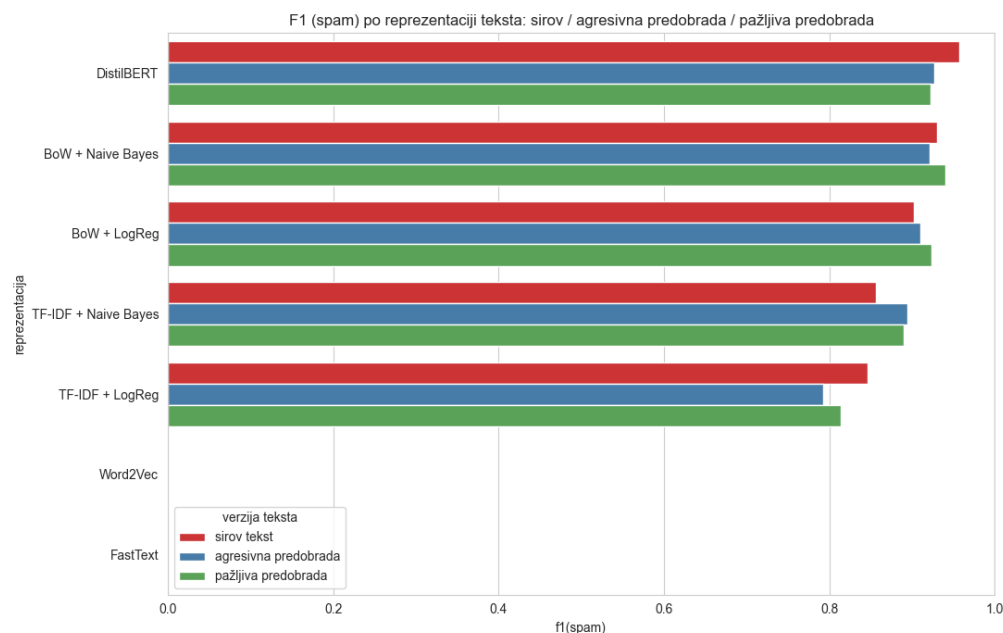
13.5 Uticaj predobrade na DistilBERT

DistilBERT, korišćen kao zamrznut ekstraktor obeležja (mean-pooling, bez fine-tuninga), daje $F1(\text{spam}) = 0,9572$ na sirovom tekstu, naspram 0,9266 (agresivna) i 0,9219 (pažljiva predobrada). DistilBERT gradi kontekstualne reprezentacije oslanjajući se na prirodni red i funkcionalne reči, pa uklanjanjem stop-reči i lematizacijom tekst gubi deo te strukture. Ni pažljivo očuvani signal ovde ne pomaže (0,9219 je čak neznatno slabije od agresivne verzije) - za razliku od BoW-a, DistilBERT nema koristi od `ALLCAPSFLAG`, jer taj marker nije deo jezika na kom je prethodno obučen, dok informaciju o velikim slovima model već čita direktno iz sirovog teksta kroz subword tokenizaciju. Strategija „sačuvaj signal kao oznaku” pomaže modelima koji inače taj signal ne vide, ali je nepotrebna kod modela koji ga već vide sami.

13.6 Uporedna analiza i matrica konfuzije

Slika 4 i tabela ispod sumiraju svih 27 testiranih kombinacija. DistilBERT na sirovom tekstu ostaje ubedljivo najbolji ($F1 = 0,9572$), ali odmah iza njega na drugom mestu nalazi se BoW + Naivni Bajes sa pažljivom predobradom ($F1 = 0,9398$), ispred sirovog teksta i ispred DistilBERT-a sa bilo kojom predobradom. Ovo je najjača pojedinačna potvrda teze rada: predobrada dizajnirana da čuva diskriminativni signal ne samo da ne šteti, već postaje druga najbolja strategija u celom eksperimentu. Unapred trenirani Word2Vec i FastText zauzimaju srednji deo ranga (13.-21. mesto) - Word2Vec-pretrained (0,785–0,794) se toliko približava TF-IDF + logističkoj regresiji (0,79–0,85) da im se rangovi doslovno isprepliću, dok FastText-pretrained (0,644–0,707) ostaje nešto slabiji. Word2Vec/FastText trenirani od nule na ovom malom korpusu ostaju na samom dnu

(F1 = 0) nezavisno od predobrade - jedina kombinacija u celom eksperimentu na koju izbor predobrade nema nikakav uticaj.



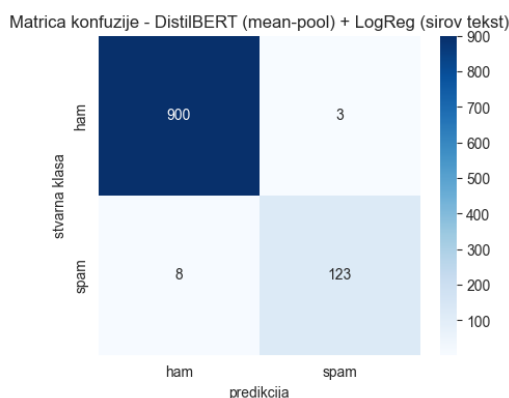
Slika 4. F1-mera (spam klasa) po reprezentaciji teksta: sirov tekst / agresivna predobrada / pažljiva predobrada.

Rang	Reprezentacija (verzija teksta)	F1(spam)
1.	DistilBERT mean-pool + LogReg (sirov tekst)	0,9572
2.	BoW + Naivni Bajes (pažljiva predobrada)	0,9398
3.	BoW + Naivni Bajes (sirov tekst)	0,9308
4.	DistilBERT mean-pool + LogReg (agresivna predobrada)	0,9266
5.	BoW + LogReg (pažljiva predobrada)	0,9231
6.-9.	DistilBERT (pažljiva), BoW+NB (agresivna), BoW+LogReg (agresivna, sirov)	0,90-0,92
10.-12.	TF-IDF + Naivni Bajes, sve tri verzije teksta	0,86-0,89
13.	TF-IDF + LogReg (sirov tekst)	0,8462
14.	TF-IDF + LogReg (pažljiva predobrada)	0,8142
15.	Word2Vec-pretrained + LogReg (pažljiva predobrada)	0,7935
16.	TF-IDF + LogReg (agresivna predobrada)	0,7928
17.	Word2Vec-pretrained + LogReg (agresivna predobrada)	0,7903
18.	Word2Vec-pretrained + LogReg (sirov tekst)	0,7851
19.	FastText-pretrained + LogReg (pažljiva predobrada)	0,7070
20.	FastText-pretrained + LogReg (agresivna predobrada)	0,6792
21.	FastText-pretrained + LogReg (sirov tekst)	0,6436
22.-27.	Word2Vec / FastText (trenirano od nule), sve tri verzije teksta	0,0000

Tabela 5. Rang svih testiranih kombinacija po F1-meri (spam klasa)

Matrica konfuzije najboljeg modela (DistilBERT mean-pool + logistička regresija, sirov tekst) na 1.034 test-poruke (903 ham, 131 spam) pokazuje: 900 ham poruka tačno prepoznato, 3 ham poruke

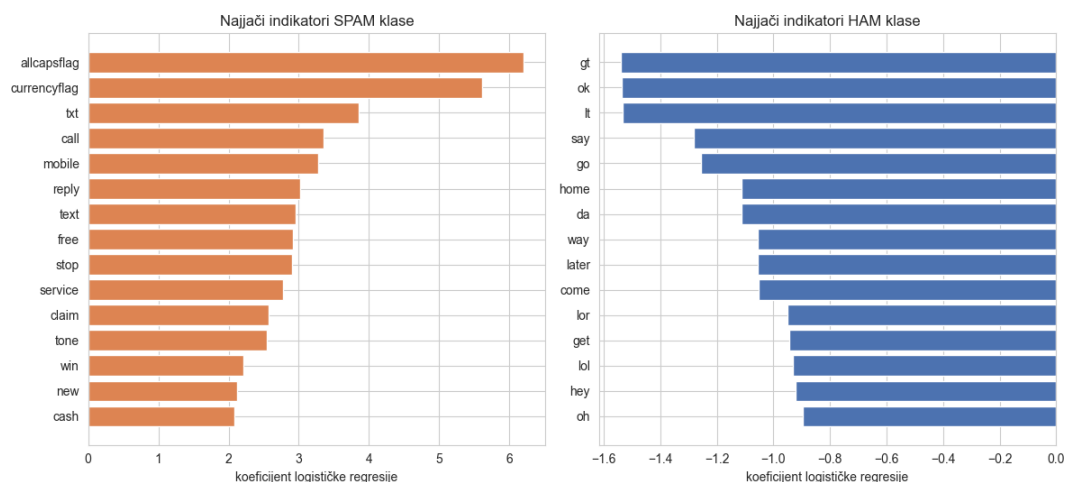
pogrešno označene kao spam (lažno pozitivni), 8 spam poruka pogrešno označeno kao ham (propušteni spam), i 123 spam poruke tačno prepoznato.



Slika 5. Matrica konfuzije najboljeg modela (DistilBERT + logistička regresija, sirov tekst).

13.7 Analiza značaja obeležja

Da bi se odgovorilo na pitanje zašto pažljiva predobrada pobeđuje kod BoW-a, trenirana je logistička regresija nad TF-IDF reprezentacijom pažljivo pripremljenog teksta, i izdvojeno je 15 reči/oznaka sa najvećim pozitivnim koeficijentom (najjači indikatori spama) i 15 sa najvećim negativnim koeficijentom (najjači indikatori ham-a).



Slika 6. Petnaest najjačih indikatora svake klase (TF-IDF + logistička regresija, pažljiva predobrada).

Rezultat je nedvosmislen: dva apsolutno najjača prediktora spama u celom modelu nisu reči, već upravo oznake dodate u pažljivoj predobradi: `ALLCAPSFLAG` i `CURRENCYFLAG`. Obe daleko ispred najjače prave reči, „txt”. Velika slova i simboli valuta jesu među najjačim signalima spama u ovom skupu, i model im upravo zato dodeljuje najveće težine čim mu se ti signali ponude kao eksplicitna obeležja umesto da budu obrisani zajedno sa ostalim šumom. Iza njih slede

očekivane reklamne reči („call”, „mobile”, „reply”, „free”, „stop”, „claim”, „cash”), u skladu sa analizom najfrekventnijih reči. Na ham strani najjači indikatori su svakodnevne, konverzijske reči („ok”, „say”, „go”, „home”, „later”, „lol”) bez promotivnih termina, što je i očekivano.

14. Diskusija i preporuke

Rezultati praktičnog dela u celini potvrđuju centralnu tezu teorijskog dela, ali je i preciziraju: predobrada teksta nije ni univerzalno korisna ni univerzalno štetna operacija, već izbor čiji ishod zavisi od toga šta se predobradom čuva, a šta briše, i za koju reprezentaciju se priprema. Kod BoW reprezentacije, pažljiva predobrada je bila najbolja od sve tri verzije kod oba klasifikatora, i analiza značaja obeležja je pokazala tačno zašto: oznake za velika slova i valutu ispale su dva najjača pojedinačna prediktora spama u celom modelu. Kod embeddinga treniranih od nule, veličina korpusa je ostala presudniji faktor od bilo kog izbora predobrade. Kod kontekstualnih transformer modela, i agresivna i pažljiva predobrada su podjednako štetile, jer oba pristupa oduzimaju kontekst (ili dodaju veštački, modelu nepoznat token) na koji se mehanizam samopažnje oslanja.

Iz ovoga proizlazi nekoliko konkretnih preporuka. Prvo, kvalitet podataka treba proveriti pre bilo koje lingvističke obrade - provera duplikata i nedostajućih vrednosti mora prethoditi podeli na trening i test skup. Drugo, i najvažnije: pre nego što se bilo koji obrazac tekstualnog šuma ukloni, treba proveriti da li je zaista slučajan ili sistematski vezan za jednu od klasa. Ako jeste, bolje ga je prevesti u eksplicitno obeležje nego obrisati. Treće, kada su raspoloživi resursi (unapred trenirani transformer model), jednostavnija predobrada ili njeno potpuno izostavljanje daje bolje rezultate; kada takvi resursi nisu dostupni, klasične retke reprezentacije uz pažljivo dizajniranu predobradu su i jeftin i efikasan izbor. Četvrto, samostalno treniranje word embeddinga ima smisla samo ako je raspoloživi korpus dovoljno velik. U suprotnom, unapred trenirani vektori ili kontekstualni modeli su bolji izbor.

15. Zaključak

Ovaj rad je pokazao da se klasični dimenzije kvaliteta podataka poput kompletnosti, konzistentnosti i prisustva šuma ne gube kod tekstualnih podataka, već poprimaju specifičan oblik uslovljen prirodom lingvističkog signala. To se prvenstveno ogleda kroz varijacije u dužini dokumenata, veličini rečnika, leksičkoj raznovrsnosti, kao i kroz specifične oblike šuma poput grešaka u kucanju, spama, HTML ostataka, emotikona i OCR grešaka. Takođe, priprema teksta se

ne može posmatrati kao skup univerzalno primenjivih pravila, već kao proces čiji efekat direktno zavisi od odabrane reprezentacije teksta i njene sposobnosti da razlikuje stvarni šum od korisnog diskriminativnog obrasca.

Praktični deo je ovu tezu potvrdio na dva nivoa. Na nivou apsolutnog rezultata: najbolji rezultat ($F1 = 0,9572$, tačnost 98,94%) postignut je najjednostavnijom kombinacijom — DistilBERT-om nad sirovim tekstom, bez ikakve predobrade. Na nivou dizajna predobrade: pokazano je da predobrada može biti drugi najbolji pristup u celom eksperimentu (BoW + Naivni Bajes sa pažljivom predobradom, $F1 = 0,9398$) - ali samo kada je eksplicitno dizajnirana da sačuva signal za koji je eksplorativna analiza pokazala da je diskriminativan. Analiza značaja obeležja je ovaj nalaz učinila opipljivim: oznake za velika slova i valutu, dodate pažljivom predobradom, ispale su jače od bilo koje pojedinačne reči u modelu. Kod embeddinga treniranih od nule, nijedan izbor predobrade nije mogao da nadoknadi nedostatak podataka.

Izbor i obim koraka pripreme teksta moraju se validirati empirijski za svaku konkretnu kombinaciju modela, reprezentacije i skupa podataka. Zbog toga se ne sme unapred pretpostaviti ni da obimnija predobrada nužno vodi do boljih performansi, niti da je predobrada podataka suvišna samo zato što određeni kompleksniji model postiže visoke rezultate nad sirovim tekstom.

16. Literatura

- [1] D. Jurafsky, J. H. Martin, "Speech and Language Processing"
<https://web.stanford.edu/~jurafsky/slp3>
- [2] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", 2018.
<https://arxiv.org/abs/1810.04805>
- [3] S. Bird, E. Klein, E. Loper, "Natural Language Processing with Python".
<https://www.nltk.org/book/>
- [4] R. Sennrich, B. Haddow, A. Birch, "Neural Machine Translation of Rare Words with Subword Units", 2015, <https://arxiv.org/abs/1508.07909>
- [5] Hugging Face, "Tokenization algorithms",
https://huggingface.co/docs/transformers/tokenizer_summary
- [6] The Unicode Consortium, "Unicode Normalization Forms", <https://unicode.org/reports/tr15/>
- [7] C. D. Manning, P. Raghavan, H. Schütze, "Introduction to Information Retrieval", Cambridge University Press, 2008, <https://nlp.stanford.edu/IR-book/>
- [8] M. F. Porter, "The Porter Stemming Algorithm", <https://tartarus.org/martin/PorterStemmer/>
- [9] Princeton University, "WordNet: A Lexical Database for English",
<https://wordnet.princeton.edu/>
- [10] Explosion AI, "Linguistic Features: Lemmatization", <https://spacy.io/usage/linguistic-features#lemmatization>
- [11] J. Wei, K. Zou, "EDA: Easy Data Augmentation Techniques for Boosting Performance on Text Classification Tasks", 2019, <https://arxiv.org/abs/1901.11196>
- [12] R. Sennrich, B. Haddow, A. Birch, "Improving Neural Machine Translation Models with Monolingual Data", 2015, <https://arxiv.org/abs/1511.06709>
- [13] scikit-learn developers, "6.2.3. Text feature extraction", scikit-learn User Guide,
https://scikit-learn.org/stable/modules/feature_extraction.html
- [14] T. Mikolov, K. Chen, G. Corrado, J. Dean, "Efficient Estimation of Word Representations in Vector Space", 2013, <https://arxiv.org/abs/1301.3781>
- [15] J. Pennington, R. Socher, C. D. Manning, "GloVe: Global Vectors for Word Representation", Stanford NLP Group, <https://nlp.stanford.edu/projects/glove/>
- [16] P. Bojanowski, E. Grave, A. Joulin, T. Mikolov, "Enriching Word Vectors with Subword Information", 2016, <https://arxiv.org/abs/1607.04606>
- [17] V. Sanh, L. Debut, J. Chaumond, T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter", 2019, <https://arxiv.org/abs/1910.01108>